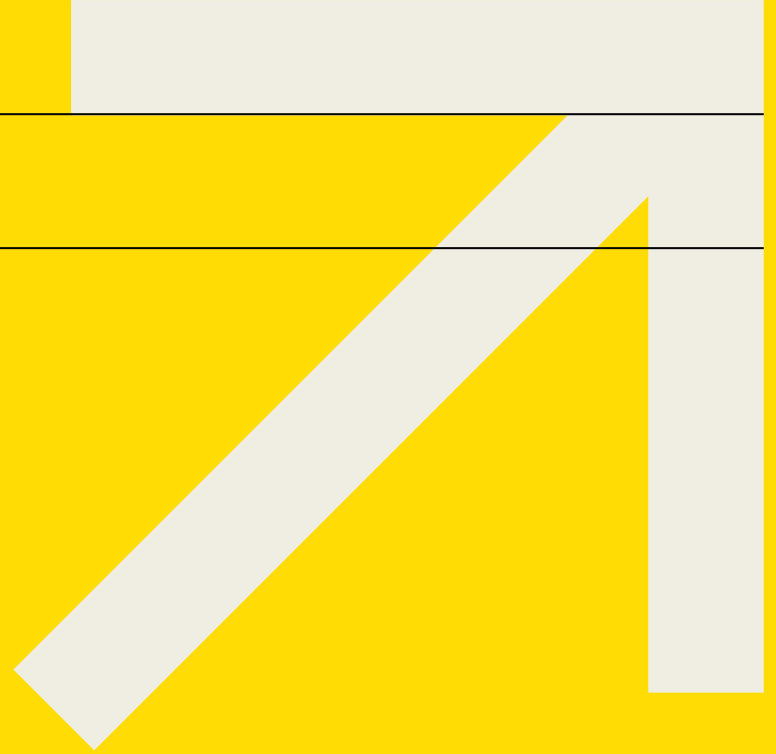


Week 5

Progress



Slide Print



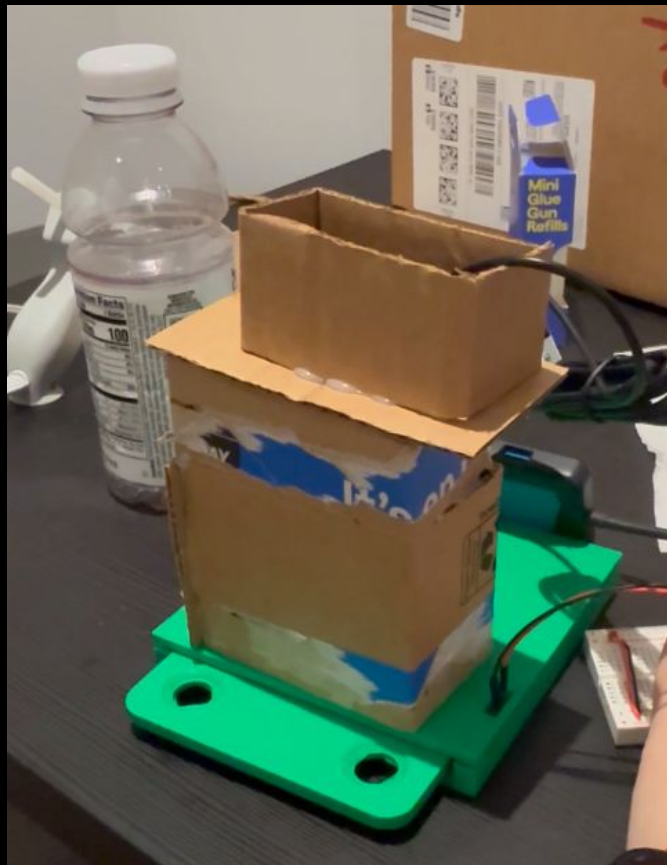
Slide Assembly



Print Backlight Housing



Testing!



Revised Image Code(Crop+Threshold)

```
PImage crop(PImage img) { //crops image from camera to get just the slide
    PImage cropped = img.get(160, 325, 640, 480);
    //cropped.save("cropped.jpg");
    return cropped;
}

float[] processImage(PImage img, PImage modImg, Boolean isWav) { //takes the image from the folder changes all red pixels to green and returns float array of drawn wave
    modImg.loadPixels(); //load display pixels
    img.loadPixels(); //load images pixels
    img = crop(img);
    img.filter(THRESHOLD, 0.795); //filters line to black
    //img.save("newimg.jpg");
    float[] wav = new float[camwidth];
```


Waveform Menu

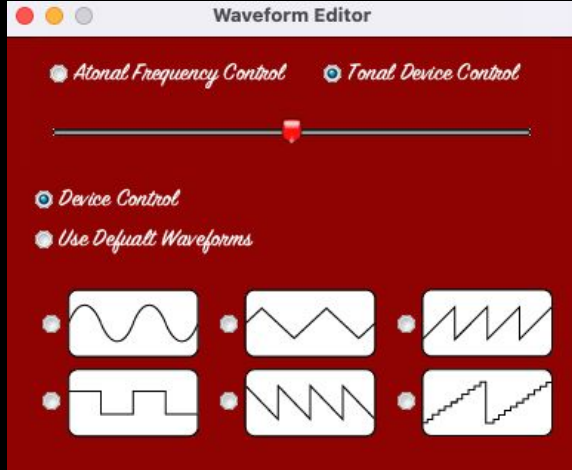
```
//Wav Generation Functions
void defaultSin() {
    defaultWav = resetSin;
}

void defaultSqr() {
    float[] sqrwav = new float[640];
    for (int i = 0; i < 640; i++) {
        if (i < 320) {
            sqrwav[i] = -1;
        } else {
            sqrwav[i] = 1;
        }
    }
    defaultWav=sqrwav;
}

void defaultTri() {
    float[] triwav = new float[640];
    int counter = 0;
    for (int i = 0; i < 640; i++) {
        if (i == 0) {
            counter = 0;
        } else if (i < 160) {
            counter +=1;
        } else if (i < 480) {
            counter -=1;
        } else {
            counter +=1;
        }
        triwav[i] = map(counter, -159, 159, 1, -1);
    }
    defaultWav=triwav;
}

void defaultSaw() {
    float[] sawwav = new float[640];
    int counter = 320;
    for (int i = 0; i < 640; i++) {
        counter -=1;
        sawwav[i] = map(counter, -319, 319, 1, -1);
    }
    defaultWav = sawwav;
}

void defaultRamp() {
    float[] rampwav = new float[640];
    int counter = -320;
    for (int i = 0; i < 640; i++) {
        counter +=1;
        rampwav[i] = map(counter, -319, 319, 1, -1);
    }
    defaultWav=rampwav;
}
```



```
//Waveform Editor
void openWavMenu() {
    if (wavOpen == false) {
        createWavGUI();
    }
}

void closingWav() {
    wavOpen = false;
}

float[] defaultWav = resetSin;

void wavDeviceControlOn() {
    wavControl = true;
    nowav.setSelected(true);
    greystuff.setVisible(true);
    wavlock.setVisible(true);
}

void wavDeviceControlOff() {
    wavControl = false;
    sinwave.setSelected(true);
    greystuff.setVisible(false);
    wavlock.setVisible(false);
}

void tonalControl() {
    sliderwavhider.setVisible(true);
    pitch.setLimits(1, 0, 2); // fix limits
    wavMult=1;
}

void atonalControl() {
    sliderwavhider.setVisible(false);
    pitch.setLimits(1, 0, 2); // fix limits
    wavMult=1;
}

void freqSlider(GCustomSlider source, GEvent event) {
    if (event == GEvent.RELEASED) {
        wavMult = source.getValueF();
    }
}
```